

INFORME TÉCNICO AVANZADO DE ARQUITECTURA DE SOFTWARE

Microservicios, Seguridad por Token, Swagger y Testing

Proyecto: VentaRapida

Framework base: Spring Boot, Spring Security, Spring Cloud Gateway, JPA, WebClient, SpringDoc
OpenAPI, JUnit 5 y Mockito

Microservicios evaluados: API Gateway, Auth, Catálogo, Cliente, Inventario, Ventas, Pagos, Facturación,
Proveedores, Reportes y Notificaciones

Nombre: Carolina Hermosilla, Antonella Olivos, Pedro Toledo

Sección: 001D

Docente: Mariangeles Robinson

Asignatura: Desarrollo Fullstack I

Fecha de entrega: 21/06/2026

Índice

Índice.....	2
1. Contextualización y contenido detallado de la Experiencia 3.....	3
1.1 Alcance técnico de la experiencia.....	3
2. Diagramas actualizados y completos.....	4
2.1 Diagrama de arquitectura actualizado.....	4
2.2 Diagrama del modelo de base de datos actualizado.....	5
3. Servicio de autenticación.....	5
3.1 ¿En qué consiste?.....	5
3.2 ¿Cómo se implementa la autenticación por token?.....	5
3.3 Spring Security, protección de rutas y CORS.....	6
4. Implementación por capas.....	9
4.1 Ejemplo de implementación por capas: service-catalogo / Producto.....	9
4.2 Evidencia de peticiones implementadas.....	10
4.3 Comunicación entre microservicios: ¿cómo se implementa?.....	12
5. Swagger / OpenAPI.....	14
5.1 ¿Qué es y en qué consiste?.....	14
5.2 ¿Cómo se implementa en microservicios Spring Boot?.....	14
5.3 Evidencia de Swagger en tres microservicios.....	15
6. Testing.....	19
6.1 ¿En qué consiste y por qué es importante testear?.....	19
6.2 Pruebas unitarias y patrón AAA.....	19
6.3 Documentación del 60% del testing implementado.....	20
6.3.1 Clase ProductoServiceTest.....	20
6.3.2 Clase CategoriaServiceTest.....	21
6.3.3 Clase ClienteServiceTest.....	22
6.3.4 Clase FacturaServiceTest.....	24
6.3.5 Clase ReporteServiceTest.....	26
6.3.6 Clase ProveedorServiceTest.....	28
6.3.7 Clase ItemCompraServiceTest.....	29
6.3.8 Clase OrdenCompraServiceTest.....	31
7. Anexo: rutas y cuerpos de prueba sugeridos.....	32
8. Conclusión.....	33

1. Contextualización y contenido detallado de la Experiencia 3

La Experiencia 3 consistió en la consolidación técnica de un ecosistema distribuido de microservicios para un sistema de Punto de Venta. El objetivo principal fue integrar servicios independientes, documentar sus APIs, proteger sus rutas mediante autenticación por token, demostrar la comunicación entre componentes y validar parte de la lógica de negocio mediante pruebas unitarias.

El sistema implementado aplica el patrón database-per-microservice, donde cada servicio mantiene su propia base de datos y expone funcionalidades mediante endpoints REST. El acceso externo se centraliza en el API Gateway, el cual redirige las solicitudes hacia cada microservicio y aplica un filtro de autenticación para las rutas operacionales.

- API Gateway: punto único de entrada, configuración de rutas, CORS global y filtro de autenticación.
- service-auth: registro, login, generación de JWT, Spring Security y cifrado de contraseñas con BCrypt.
- service-catalogo: administración de categorías y productos, más endpoint de existencia de producto para otros servicios.
- service-cliente: administración de clientes y actualización parcial de datos de contacto y estado.
- service-inventario: gestión de stock, movimientos de inventario, alertas por bajo stock y enriquecimiento de productos mediante WebClient.
- service-ventas: gestión de órdenes y detalles de venta, cálculo de total recaudado e integración con productos, inventario, pagos y facturación.
- service-pagos: registro de pagos, métodos de pago, estados internos e integración con Flow Sandbox.
- service-facturacion: generación y consulta de facturas asociadas a órdenes de venta.
- service-proveedores: proveedores, órdenes de compra, ítems de compra y registro de entradas de inventario al recibir una orden.
- service-reportes: generación de reportes consolidados consumiendo datos de ventas e inventario.
- service-notificacion: registro y consulta de notificaciones del sistema.

1.1 Alcance técnico de la experiencia

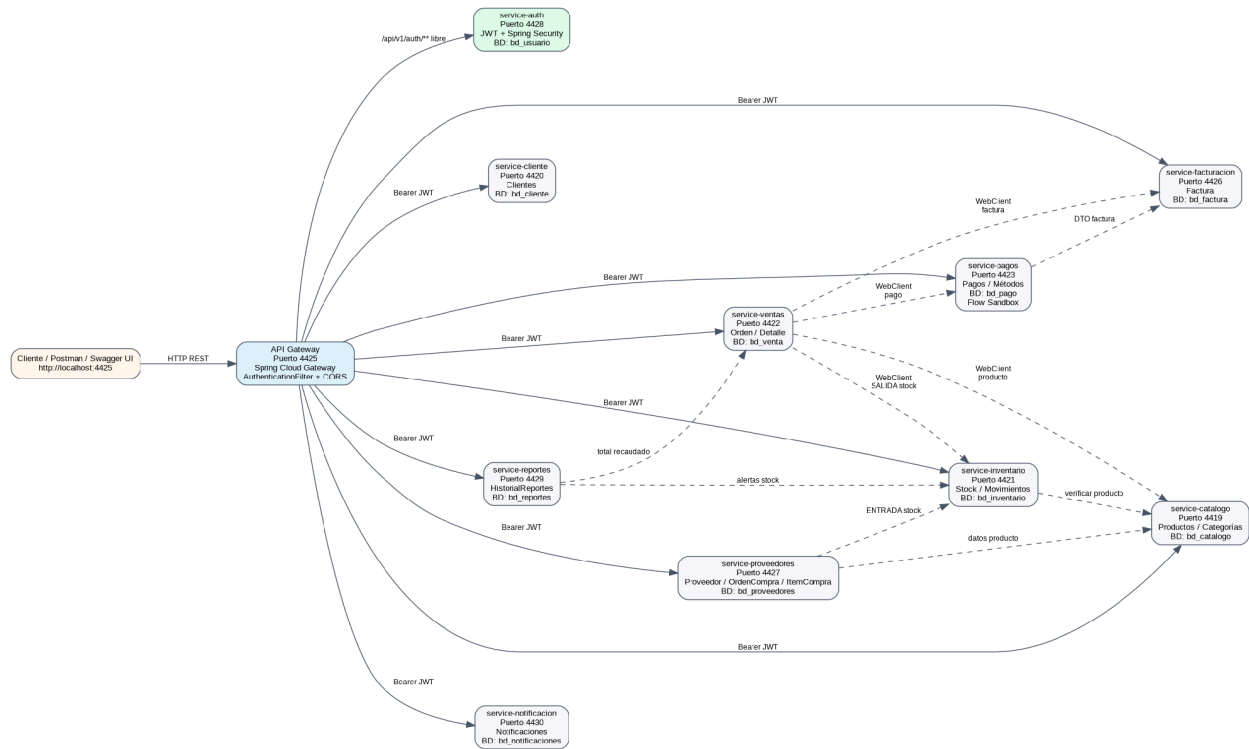
El alcance de esta experiencia incluyó la implementación de endpoints CRUD o de consulta/actualización en servicios principales, la documentación con Swagger/OpenAPI, la configuración de rutas protegidas en el gateway y la creación de pruebas unitarias con JUnit 5 y Mockito. Además, se documentó la comunicación entre microservicios mediante WebClient, especialmente en los flujos de inventario, ventas, proveedores y reportes.

Componente	Puerto	Base de datos	Responsabilidad principal
api-gateway	4425	No aplica	Entrada única, rutas, CORS y filtro JWT
service-auth	4428	bd_usuario	Login, registro, roles y generación de JWT
service-catalogo	4419	bd_catalogo	Productos y categorías
service-cliente	4420	bd_cliente	Clientes
service-inventario	4421	bd_inventario	Stock y movimientos
service-ventas	4422	bd_venta	Órdenes y detalles de venta
service-pagos	4423	bd_pago	Pagos, métodos de pago e integración Flow
service-facturacion	4426	bd_factura	Facturas
service-proveedores	4427	bd_proveedores	Compras y proveedores
service-reportes	4429	bd_reportes	Historial y reportes consolidados
service-notificacion	4430	bd_notificaciones	Notificaciones

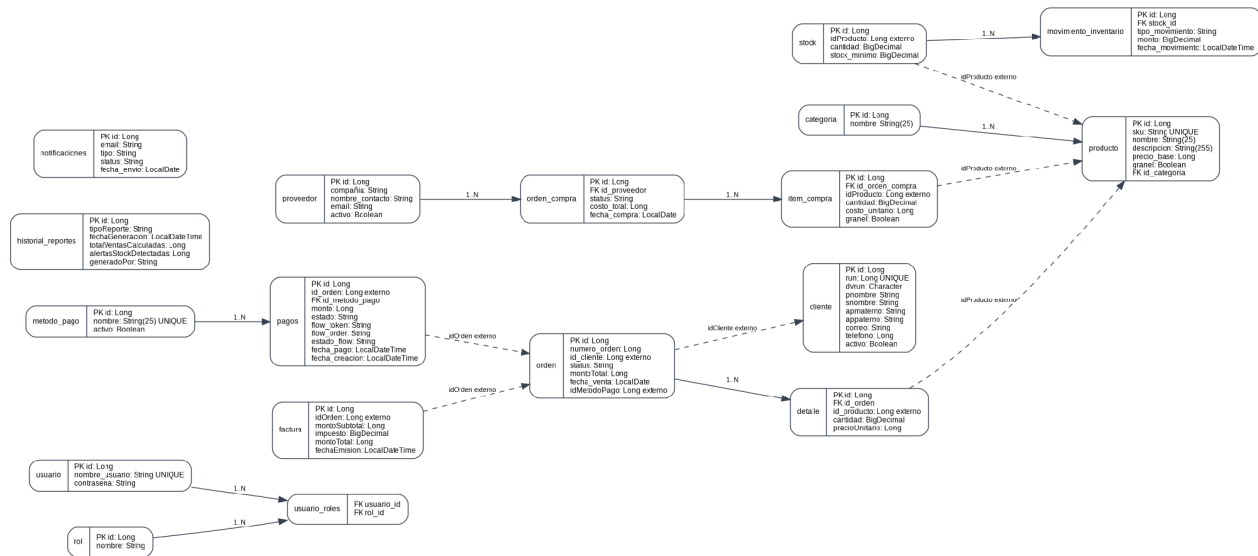
2. Diagramas actualizados y completos

Los siguientes diagramas representan el estado actual del proyecto revisado. Se incluyen como base para el informe, pero se deja espacio para reemplazarlos por capturas exportadas desde la herramienta exigida por el docente, si corresponde.

2.1 Diagrama de arquitectura actualizado



2.2 Diagrama del modelo de base de datos actualizado



El modelo muestra las entidades persistentes detectadas en los microservicios. Las relaciones sólidas representan asociaciones JPA internas del mismo microservicio; las relaciones punteadas representan referencias lógicas por ID hacia datos administrados por otro microservicio, por ejemplo idProducto, idCliente o idOrden.

3. Servicio de autenticación

3.1 ¿En qué consiste?

El servicio de autenticación centraliza el proceso de identificación de usuarios dentro del sistema. En vez de que cada microservicio valide usuario y contraseña por separado, el microservicio service-auth recibe las credenciales, valida los datos contra la base bd_usuario y genera un token JWT. Este token se utiliza después para acceder a las rutas protegidas del API Gateway.

En este proyecto, las entidades principales del servicio son Usuario y Rol. Usuario se mapea a la tabla usuario, Rol se mapea a la tabla rol y la relación entre ambos se representa mediante la tabla intermedia usuario_roles. Esto permite asignar uno o más roles a cada usuario autenticado.

3.2 ¿Cómo se implementa la autenticación por token?

1. El cliente envía una petición POST a /api/v1/auth/login con nombreUsuario y contraseña.
2. AuthService busca el usuario mediante UsuarioRepository.findByNombreUsuario.
3. La contraseña ingresada se compara con la contraseña cifrada usando PasswordEncoder y BCrypt.
4. Si las credenciales son válidas, JwtService genera un JWT firmado con HS256, sujeto username, roles del usuario y expiración de dos horas.
5. El cliente utiliza el token en cada petición protegida mediante el header Authorization: Bearer <token>.
6. El API Gateway intercepta la solicitud con AuthenticationFilter y valida la firma del token usando la misma clave secreta configurada en application.yml.

```
POST http://localhost:4425/api/v1/auth/login
Content-Type: application/json
```

```
{
  "nombreUsuario": "servicios",
  "contrasena": "Servicios123"
}
```

3.3 Spring Security, protección de rutas y CORS

Spring Security se configura en service-auth mediante SecurityConfig. En esta clase se deshabilita CSRF para trabajar con APIs REST, se liberan las rutas de autenticación y documentación Swagger, y se exige autenticación para cualquier otra solicitud. También se define PasswordEncoder con BCryptPasswordEncoder para almacenar contraseñas cifradas.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http.csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth.requestMatchers(
            "/api/v1/auth/**",
            "/swagger-ui/**",
            "/swagger-ui.html",
            "/v3/api-docs/**"
        ).permitAll().anyRequest().authenticated())
        .build();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

La protección de rutas operacionales se realiza principalmente en el API Gateway. En application.yml, la ruta /api/v1/auth/** queda libre para login y registro, mientras que rutas como /api/v1/productos/**, /api/v1/clientes/**, /api/v1/stock/**, /api/v1/ordenes/**, /api/v1/proveedores/**, /api/v1/facturas/** y /api/v1/reportes/** aplican AuthenticationFilter. Si el header no existe, no comienza con Bearer o el token no es válido, el gateway responde 401 Unauthorized.

CORS se configura tanto en el API Gateway como en los microservicios para permitir llamadas desde el origen del gateway y aceptar métodos GET, POST, PUT, PATCH, DELETE y OPTIONS. Esto evita bloqueos del navegador cuando Swagger UI o un cliente web realizan peticiones cruzadas entre puertos.

EVIDENCIA: CÓDIGO DE SECURITYCONFIG, JWTSERVICE Y AUTHENTICATIONFILTER

```

@Configuration
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception{
        return http.csrf(csrf -> csrf.disable()).authorizeHttpRequests(auth -> auth.requestMatchers(
            "/api/v1/auth/**",
            "/swagger-ui/**",
            "/swagger-ui.html",
            "/v3/api-docs/**").permitAll().anyRequest().authenticated()).build();
    }
    @Bean
    public PasswordEncoder passwordEncoder(){
        return new BCryptPasswordEncoder();
    }
}

```

```

@Service
public class JwtService {
    @Value("${jwt.secret}")
    private String secreto;
    public String generarToken(String username, List<String> rol){
        long dosHorasms = 1000L * 60 * 60 * 2;
        return Jwts.builder()
            .setSubject(username)
            .claim("rol", rol)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis()+dosHorasms))
            .signWith(Keys.hmacShaKeyFor(secreto.getBytes()), SignatureAlgorithm.HS256)
            .compact();
    }
}

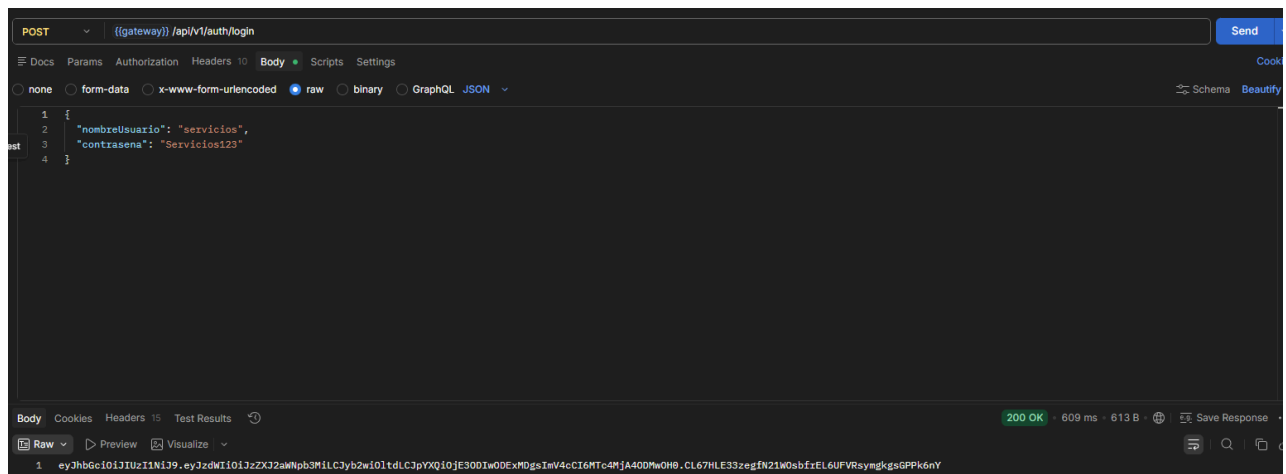
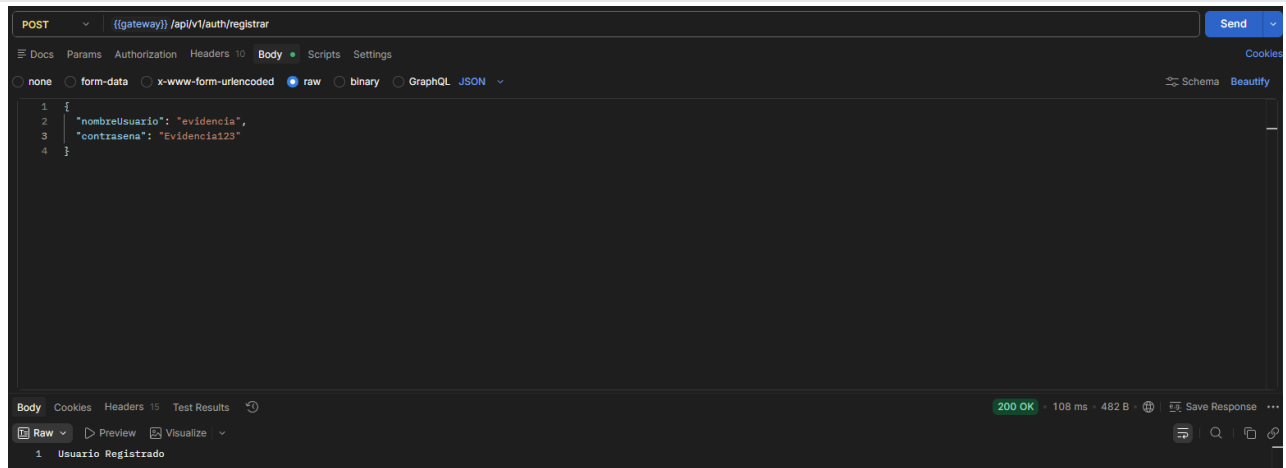
```

```

@Component
public class AuthenticationFilter extends AbstractGatewayFilterFactory<AuthenticationFilter.Config>{
    @Value("${jwt.secret}")
    private String secreto;
    public AuthenticationFilter(){
        super(Config.class);
    }
    public static class Config{
    }
    @Override
    public GatewayFilter apply(Config config){
        return (exchange, chain) ->{
            String authHeader = exchange.getRequest().getHeaders().getFirst(HttpHeaders.AUTHORIZATION);
            //validar existencia de header con formato correcto
            if(authHeader==null || !authHeader.startsWith(prefix: "Bearer ")){
                return onError(exchange, err: "Token faltante o formato inválido", HttpStatus.UNAUTHORIZED);
            }
            String token = authHeader.substring(beginIndex: 7);

            try {
                //validar token
                Jwts.parserBuilder().setSigningKey(Keys.hmacShaKeyFor(secreto.getBytes(StandardCharsets.UTF_8))).build().parseClaimsJws(token);
            } catch (Exception e) {
                //token expirado o firma erronea
                return onError(exchange, err: "Token inválido o expirado", HttpStatus.UNAUTHORIZED);
            }
            return chain.filter(exchange);
        };
    }
    private Mono<Void> onError(ServerWebExchange exchange, String err, HttpStatus httpStatus){
        exchange.getResponse().setStatusCode(httpStatus);
        return exchange.getResponse().setComplete();
    }
}

```


EVIDENCIA: EJECUCIÓN LOGIN Y USO DEL TOKEN EN POSTMAN

4. Implementación por capas

La implementación por capas separa responsabilidades y facilita el mantenimiento. En este proyecto se utiliza una estructura común en cada microservicio: model, repository, service, controller, config y exception. La capa Model define entidades JPA; Repository accede a la base de datos; Service concentra las reglas de negocio; Controller expone endpoints REST; Config contiene CORS, Swagger, WebClient o seguridad; Exception centraliza respuestas de error.

4.1 Ejemplo de implementación por capas: service-catalogo / Producto

Como ejemplo se documenta el flujo de Producto en service-catalogo, porque incluye operaciones POST, GET, GET por ID, PUT y DELETE. La entidad Producto se encuentra en model y contiene

campos como sku, nombre, descripcion, precioBase, granel y categoria. ProductoRepository extiende JpaRepository y permite usar operaciones como save, findAll, findById y delete. ProductoService aplica la lógica de negocio, por ejemplo validar existencia antes de actualizar o eliminar. ProductoController expone las rutas REST bajo /api/v1/productos y retorna respuestas con ResponseEntity.

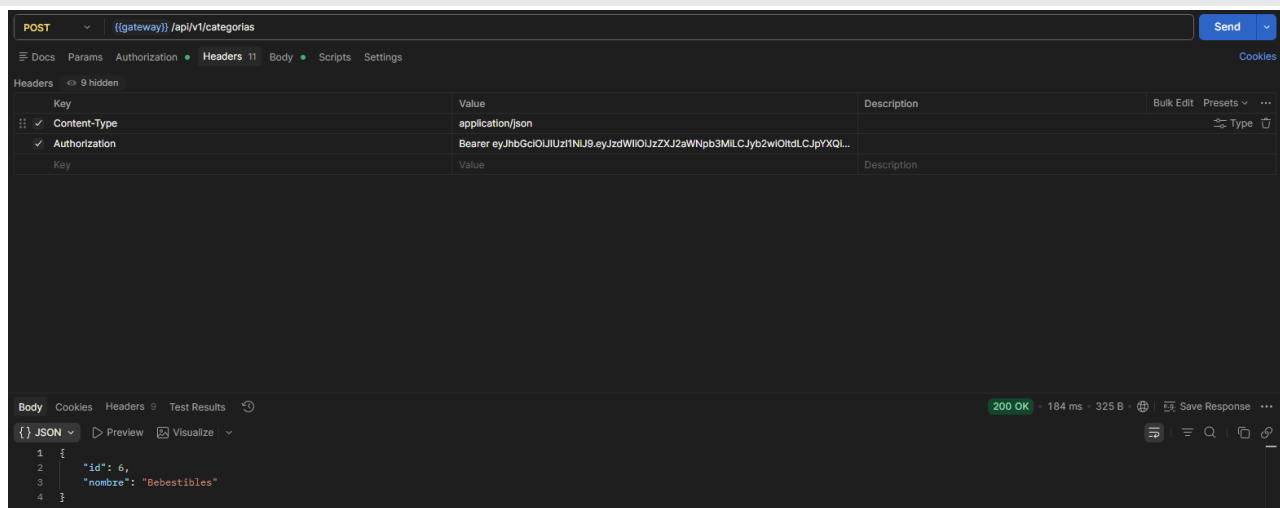
Capa	Archivo / ruta	Responsabilidad
Model	service-catalogo/src/main/java/.../model/Producto.java	Representa tabla producto y relación con categoria.
Repository	service-catalogo/src/main/java/.../repository/ProductoRepository.java	Abstrae acceso a datos con JpaRepository.
Service	service-catalogo/src/main/java/.../service/ProductoService.java	Contiene crear, listar, buscar, actualizar, eliminar y existe.
Controller	service-catalogo/src/main/java/.../controller/ProductoController.java	Expone endpoints REST /api/v1/productos.
Exception	service-catalogo/src/main/java/.../exception/GlobalExceptionHandler.java	Estandariza errores cuando se lanza RuntimeException.

4.2 Evidencia de peticiones implementadas

Se adjuntan capturas tanto de codificación como de ejecución en Postman. Para pruebas individuales se recomienda usar el gateway para probar rutas protegidas: `http://localhost:4425/api/v1/productos`, agregando el header Authorization con el token obtenido desde `/api/v1/auth/login`.

Operación	Endpoint sugerido	Body sugerido / observación
POST	POST <code>http://localhost:4425/api/v1/productos</code>	<code>{"sku":"ARR-001","nombre":"Arroz","descripcion":"Arroz 1kg","precioBase":1200,"granel":false,"categoria":{"id":1}}</code>
GET	GET <code>http://localhost:4425/api/v1/productos</code>	No requiere body. Debe devolver lista JSON.
GET(id)	GET <code>http://localhost:4425/api/v1/productos/1</code>	No requiere body. Debe devolver un producto existente.
PUT(id)	PUT <code>http://localhost:4425/api/v1/productos/1</code>	<code>{"sku":"ARR-001","nombre":"Arroz actualizado","descripcion":"Arroz grano largo 1kg","precioBase":1300,"granel":false,"categoria":{"id":1}}</code>
DELETE(id)	DELETE <code>http://localhost:4425/api/v1/productos/1</code>	No requiere body. Debe devolver mensaje de eliminación.

EVIDENCIA: CÓDIGO Y POSTMAN POST - creación



EVIDENCIA: CÓDIGO Y POSTMAN GET - listar todos

Postman interface showing a GET request to `/api/v1/categorias` with a Bearer token. The response is a 200 OK status with a JSON array of 6 categories.

Key	Value	Description
Content-Type	application/json	
Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXQL...	

```
1 [
2   {
3     "id": 1,
4     "nombre": "Abarrotes"
5   },
6   {
7     "id": 2,
8     "nombre": "Limpieza"
9   },
10  {
11    "id": 3,
12    "nombre": "Bebidas"
13  },
14  {
15    "id": 4,
16    "nombre": "Lacteos"
17  },
18  {
19    "id": 5,
20    "nombre": "Frutas y Verduras"
21  },
22  {
23    "id": 6,
24    "nombre": "Rebestibles"
25  }
26 ]
```

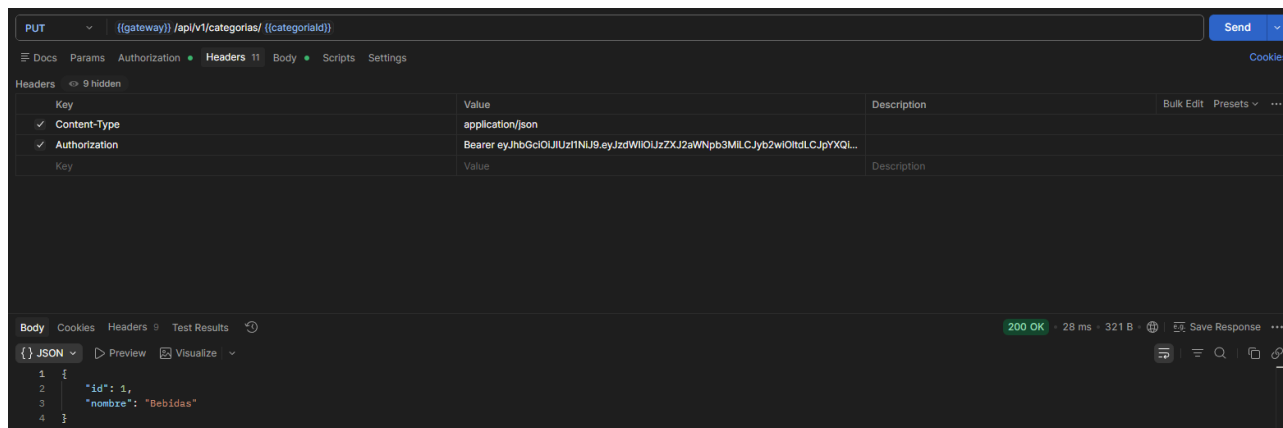
EVIDENCIA: CÓDIGO Y POSTMAN GET por ID - consulta específica

Postman interface showing a GET request to `/api/v1/categorias/{categoryId}` with a Bearer token. The response is a 200 OK status with a JSON object for category ID 1.

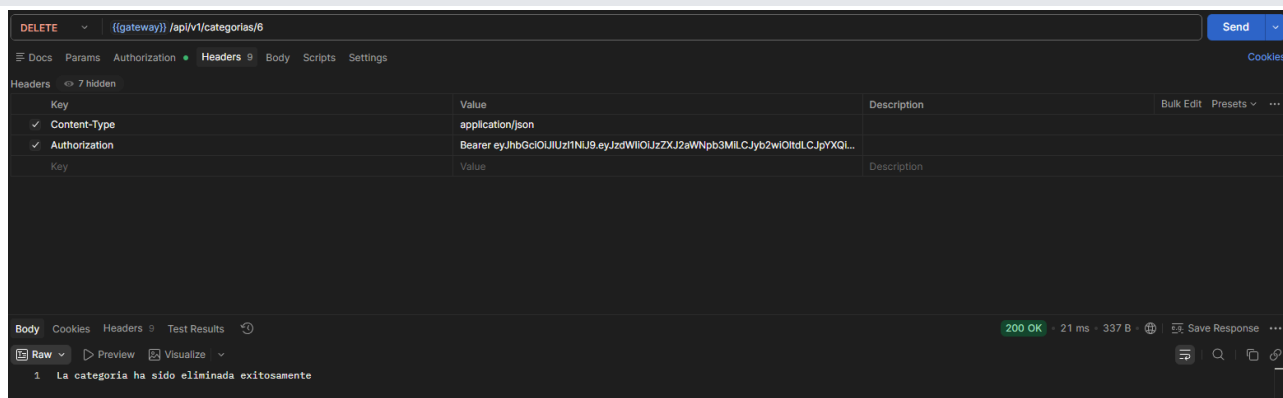
Key	Value	Description
Content-Type	application/json	
Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXQL...	

```
1 {
2   "id": 1,
3   "nombre": "Abarrotes"
4 }
```

EVIDENCIA: CÓDIGO Y POSTMAN PUT por ID - actualización



EVIDENCIA: CÓDIGO Y POSTMAN DELETE por ID - eliminación



4.3 Comunicación entre microservicios: ¿cómo se implementa?

La comunicación entre microservicios se implementa con WebClient de Spring WebFlux. Cada servicio que necesita consumir datos externos declara un WebClient.Builder en una clase de configuración y luego lo inyecta en la capa Service. El consumo se realiza mediante una cadena fluida: `build()`, `get()` o `post()`, `uri()`, `bodyValue()` cuando corresponde, `retrieve()`, `bodyToMono()` y `block()`.

En service-proveedores existe una implementación adicional: `ServiceTokenProvider` obtiene automáticamente un token desde el endpoint de login usando credenciales internas. Luego, `WebClientConfig` agrega un filtro que incorpora el header `Authorization: Bearer <token>` en cada llamada saliente. Esto permite que las llamadas internas hacia endpoints protegidos del gateway incluyan token sin tener que escribir manualmente el header en cada método.

- service-inventario consulta service-catalogo para verificar y enriquecer datos de producto.
- service-proveedores consulta catálogo para cargar datos de producto y registra movimientos ENTRADA en inventario cuando una orden pasa a RECIBIDA.
- service-reportes consulta ventas /total-recaudado e inventario /alertas-count para generar reportes consolidados.
- service-ventas se integra con catálogo, inventario, pagos y facturación para orquestar el flujo de venta.

EVIDENCIA: WEBCLIENT Y TOKEN INTERNO ENTRE MICROSERVICIOS

```

1  package com.punto_de_venta.service_proveedores.config;
2
3  import org.springframework.context.annotation.Bean;
4  import org.springframework.context.annotation.Configuration;
5  import org.springframework.http.HttpHeaders;
6  import org.springframework.web.reactive.function.client.ClientRequest;
7  import org.springframework.web.reactive.function.client.WebClient;
8
9  @Configuration
10 public class WebClientConfig {
11
12     @Bean
13     public WebClient.Builder webClientBuilder(ServiceTokenProvider serviceTokenProvider) {
14         return WebClient.builder()
15             .filter((request, next) -> {
16                 ClientRequest newRequest = ClientRequest
17                     .from(request)
18                     .header(HttpHeaders.AUTHORIZATION, "Bearer " + serviceTokenProvider.getToken())
19                     .build();
20
21                 return next.exchange(newRequest);
22             });
23     }
24 }

```

```

public class ServiceTokenProvider {

    @Value("${internal.auth.login-url}")
    private String loginUrl;

    @Value("${internal.auth.username}")
    private String username;

    @Value("${internal.auth.password}")
    private String password;

    private String token;
    private LocalDateTime tokenExpiration = LocalDateTime.now();

    public String getToken() {
        if (token == null || LocalDateTime.now().isAfter(tokenExpiration)) {
            token = login();
            tokenExpiration = LocalDateTime.now().plusMinutes(minutes: 110);
        }

        return token;
    }
}

```

```

@Transactional
public Orden procesarVenta(Orden orden){
    Long totalOrden = 0L;

    for (Detalle detalle : orden.getDetalles()) {
        detalle.setOrden(orden);

        ProductoDTO prod = webClientBuilder.build()
            .get().uri("http://localhost:4425/api/v1/productos/"+detalle.getIdProducto())
            .retrieve()
            .bodyToMono(ProductoDTO.class).block();

        if (prod != null) {
            detalle.setPrecioUnitario(prod.getPrecioBase());
            detalle.setDatosProducto(prod);
        }

        webClientBuilder.build()
            .post()
            .uri("http://localhost:4425/api/v1/movimiento-inventario/nuevo/"+detalle.getIdProducto())
            .bodyValue(new MovimientoDTO(tipoMovimiento: "Salida", detalle.getCantidad(), LocalDateTime.now()))
            .retrieve()
            .onStatus(HttpStatus::isError, response -> {
                return Mono.error(new RuntimeException(message: "Error al actualizar inventario"));
            }).bodyToMono(Void.class).block();

        Long precio = (detalle.getPrecioUnitario() != null) ? detalle.getPrecioUnitario() : 0L;

        totalOrden+=Math.round(precio* detalle.getCantidad().doubleValue());
    }
    orden.setMontoTotal(totalOrden);
    orden.setFechaVenta(LocalDate.now());
    orden.setStatus(status: "Pendiente");

    System.out.println("DEBUG: id pago: "+orden.getIdMetodoPago());
}

```

5. Swagger / OpenAPI

5.1 ¿Qué es y en qué consiste?

Swagger, actualmente asociado al estándar OpenAPI, permite documentar APIs REST de forma automática e interactiva. En vez de mantener documentación manual separada, SpringDoc analiza controladores, modelos, anotaciones y rutas para generar un contrato de API en formato JSON y una interfaz web Swagger UI. Desde esta interfaz se pueden revisar endpoints, parámetros, cuerpos de solicitud, esquemas de respuesta y probar peticiones directamente desde el navegador.

5.2 ¿Cómo se implementa en microservicios Spring Boot?

1. Agregar la dependencia springdoc-openapi-starter-webmvc-ui en el pom.xml de cada microservicio MVC.
2. Configurar en application.properties la ruta del JSON OpenAPI con springdoc.api-docs.path.
3. Configurar Swagger UI con springdoc.swagger-ui.path=/swagger-ui.html.
4. Agregar una clase OpenApiConfig para definir título, descripción, versión y servidor base.
5. Anotar los controladores con @Tag, @Operation y @ApiResponses para que la documentación sea más clara.
6. En el API Gateway, registrar rutas hacia cada /v3/api-docs para poder centralizar la documentación si se requiere.

```
# Ejemplo service-catalogo
springdoc.api-docs.path=/api/v1/catalogo/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
spring.mvc.cors.allowed-origins=http://localhost:4425
spring.mvc.cors.allowed-methods=GET,POST,PUT,DELETE,OPTIONS
spring.mvc.cors.allowed-headers=*
```

5.3 Evidencia de Swagger en tres microservicios

Microservicio	URL directa Swagger UI	JSON OpenAPI mediante gateway
Catálogo	http://localhost:4419/swagger-ui.html	http://localhost:4425/api/v1/catalogo/v3/api-docs
Inventario	http://localhost:4421/swagger-ui.html	http://localhost:4425/api/v1/inventario/v3/api-docs
Proveedores	http://localhost:4427/swagger-ui.html	http://localhost:4425/api/v1/proveedores/v3/api-docs
Facturación	http://localhost:4426/swagger-ui.html	http://localhost:4425/api/v1/facturas/v3/api-docs
Reportes	http://localhost:4429/swagger-ui.html	http://localhost:4425/api/v1/reportes/v3/api-docs

EVIDENCIA: SWAGGER UI - MICROSERVICIO CATÁLOGO

```
spring.application.name=service-catalogo
spring.datasource.url=jdbc:mysql://localhost:3306/bd_catalogo
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

server.port=4419

# Ruta donde se generará el JSON de la documentación
springdoc.api-docs.path=/api/v1/catalogo/v3/api-docs

# Ruta de Swagger UI
springdoc.swagger-ui.path=/swagger-ui.html

# Configuración CORS para permitir peticiones desde el api-gateway
spring.mvc.cors.allowed-origins=http://localhost:4425
spring.mvc.cors.allowed-methods=GET,POST,PUT,DELETE,OPTIONS
spring.mvc.cors.allowed-headers=*
```

http://localhost:4425 - Servidor a través del Gateway

Authorize

Gestión de Categorías

Endpoints para administrar las categorías del catálogo

GET	/api/v1/categorias/{id}	Buscar categoría por ID	🔒	▼
PUT	/api/v1/categorias/{id}	Actualizar categoría	🔒	▼
DELETE	/api/v1/categorias/{id}	Eliminar categoría	🔒	▼
GET	/api/v1/categorias	Listar categorías	🔒	▼
POST	/api/v1/categorias	Crear categoría	🔒	▼

Gestión de Productos

Endpoints para administrar los productos del catálogo

GET	/api/v1/productos/{id}	Buscar producto por ID	🔒	▼
PUT	/api/v1/productos/{id}	Actualizar producto	🔒	▼
DELETE	/api/v1/productos/{id}	Eliminar producto	🔒	▼
GET	/api/v1/productos	Listar productos	🔒	▼
POST	/api/v1/productos	Crear producto	🔒	▼
GET	/api/v1/productos/sku/{sku}	Buscar producto por SKU	🔒	▼
GET	/api/v1/productos/existe/{id}	Verificar existencia de producto	🔒	▼

EVIDENCIA: SWAGGER UI - MICROSERVICIO INVENTARIO


```

spring.application.name=service-inventario

spring.datasource.url=jdbc:mysql://localhost:3306/bd_inventario
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

server.port=4421

springdoc.api-docs.path=/api/v1/inventario/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html

spring.mvc.cors.allowed-origins=http://localhost:4425
spring.mvc.cors.allowed-methods=GET,POST,PUT,PATCH,DELETE,OPTIONS
spring.mvc.cors.allowed-headers=*

internal.auth.login-url=http://localhost:4425/api/v1/auth/login
internal.auth.username=servicios
internal.auth.password=Servicios123

```

http://localhost:4425 - Servidor de acceso unificado (API Gateway)		Authorize
Gestión de Inventario y Stock Endpoints para consultar la disponibilidad de productos, establecer límites y visualizar alertas de reposición		
PATCH	/api/v1/stock/stock-minimo/{id}	Actualizar nivel de stock mínimo
GET	/api/v1/stock	Listar inventario completo
GET	/api/v1/stock/{id}	Obtener stock por ID
GET	/api/v1/stock/bajo-stock	Listar productos con bajo stock
GET	/api/v1/stock/alertas-count	Contador de alertas de inventario
Historial de Movimientos Endpoints para registrar y consultar entradas y salidas de inventario		
POST	/api/v1/movimiento-inventario/nuevo/{id}	Registrar nuevo movimiento
GET	/api/v1/movimiento-inventario	Listar todos los movimientos
GET	/api/v1/movimiento-inventario/{id}	Buscar movimiento por ID
GET	/api/v1/movimiento-inventario/entre/{ini}/y/{fin}	Buscar movimientos entre fechas
GET	/api/v1/movimiento-inventario/buscar/{dia}	Buscar movimientos por día
GET	/api/v1/movimiento-inventario/buscar-desde/{fecha}	Buscar movimientos a partir de una fecha

EVIDENCIA: SWAGGER UI - MICROSERVICIO PROVEEDORES

```
1  spring.application.name=service_proveedores
2  spring.datasource.url=jdbc:mysql://localhost:3306/bd_proveedores
3  spring.datasource.username=root
4  spring.datasource.password=
5  spring.jpa.hibernate.ddl-auto=update
6  spring.jpa.show-sql=true
7
8  server.port=4427
9
10 # Ruta donde se generará el JSON de la documentación
11 springdoc.api-docs.path=/api/v1/proveedores/v3/api-docs
12
13 # Ruta de Swagger UI
14 springdoc.swagger-ui.path=/swagger-ui.html
15
16 # Configuración CORS para permitir peticiones desde el api-gateway
17 spring.mvc.cors.allowed-origins=http://localhost:4425
18 spring.mvc.cors.allowed-methods=GET,POST,PUT,PATCH,DELETE,OPTIONS
19 spring.mvc.cors.allowed-headers=*
20
21 internal.auth.login-url=http://localhost:4425/api/v1/auth/login
22 internal.auth.username=servicios
23 internal.auth.password=Servicios123
```

http://localhost:4425 - Servidor a través del Gateway

Authorize

Gestión de Órdenes de Compra Endpoints para crear, listar, buscar y actualizar órdenes de compra

GET /api/v1/ordenes-compras Listar órdenes de compra

POST /api/v1/ordenes-compras Crear orden de compra

PATCH /api/v1/ordenes-compras/cambiar-status/{id} Actualizar estado de orden de compra

GET /api/v1/ordenes-compras/{id} Buscar orden de compra por ID

Gestión de Proveedores Endpoints para registrar, listar, buscar y actualizar proveedores

GET /api/v1/proveedores Listar proveedores

POST /api/v1/proveedores Crear proveedor

GET /api/v1/proveedores/{id} Buscar proveedor por ID

PATCH /api/v1/proveedores/{id} Actualizar proveedor

PATCH /api/v1/proveedores/cambiar-status/{id} Actualizar estado del proveedor

Gestión de Ítems de Compra Endpoints para crear, listar y buscar ítems de compra

GET /api/v1/items-compras Listar ítems de compra

POST /api/v1/items-compras Crear ítem de compra

GET /api/v1/items-compras/{id} Buscar ítem de compra por ID

6. Testing

6.1 ¿En qué consiste y por qué es importante testear?

El testing consiste en ejecutar pruebas controladas para comprobar que el sistema cumple el comportamiento esperado. En un proyecto de microservicios, testear es importante porque cada servicio evoluciona de forma independiente y una modificación en la lógica de negocio puede romper funcionalidades ya existentes. Las pruebas automatizadas ayudan a detectar errores temprano, reducen regresiones y entregan evidencia objetiva del correcto funcionamiento del código.

6.2 Pruebas unitarias y patrón AAA

Las pruebas unitarias verifican una unidad pequeña del sistema, normalmente un método de la capa Service, de manera aislada. Para evitar depender de una base de datos real o de otro microservicio, se usan mocks con Mockito. El patrón AAA ordena cada prueba en tres momentos: Arrange, Act y Assert.

Fase	Significado	Ejemplo en el proyecto
Arrange	Preparar datos de entrada y comportamiento de mocks.	when(productoRepository.findById(1L)).thenReturn(Optional.of(producto));
Act	Ejecutar el método que se está probando.	Producto resultado = productoService.buscarProductoId(1L);

Assert	Verificar resultados y llamadas esperadas.	<code>assertEquals("Arroz", resultado.getNombre()); verify(repository).findByld(1L);</code>
--------	--	---

6.3 Documentación del 60% del testing implementado

A continuación se documenta una selección amplia de pruebas implementadas. La evidencia debe cubrir al menos el 60% de los métodos testeados. En el proyecto revisado existen pruebas para Catálogo, Cliente, Facturación, Notificaciones, Reportes y Proveedores. Se recomienda adjuntar capturas de las clases de test y del resultado en verde de JUnit o Maven.

Clase de test	Métodos documentados	Qué valida
CategoriaServiceTest	5	Crear, listar, buscar por ID, actualizar y eliminar categoría.
ProductoServiceTest	8	Crear, listar, buscar por ID/SKU, actualizar, eliminar y verificar existencia.
ClienteServiceTest	6	Crear, listar, buscar por ID/RUN, actualizar datos y actualizar estado.
FacturaServiceTest	5	Cálculo de subtotal/impuesto, listado, orden cronológico y búsqueda por ID.
NotificacionServiceTest	4	Crear, buscar por ID, error al no existir y eliminar.
ReporteServiceTest	2	Obtener historial y generar reporte consolidado con WebClient mockeado.
ProveedorServiceTest	6	Crear, listar, buscar, error, actualizar datos y estado.
ItemCompraServiceTest	4	Subtotal, costo total de orden, listado y búsqueda por ID.
OrdenCompraServiceTest	3	Crear orden, listar con WebClient y cambiar estado con movimiento inventario.

6.3.1 Clase ProductoServiceTest

Valida el servicio de productos usando ProductoRepository como mock. Se testean operaciones CRUD y la existencia de productos. En crear producto se prepara una categoría y producto simulado, se mockea save y se verifica que el resultado no sea nulo y mantenga los datos esperados.

EVIDENCIA: CODIFICACIÓN ProductoServiceTest

```

24 @ExtendWith(MockitoExtension.class)
25 public class ProductoServiceTest {
26
27     @Mock
28     private ProductoRepository productoRepository;
29
30     @InjectMocks
31     private ProductoService productoService;
32
33     @Test
34     @DisplayName("Debería crear un producto correctamente")
35     void guardarProductoTest() {
36
37         Categoria categoria = new Categoria();
38         categoria.setId(id: 1L);
39         categoria.setNombre(nombre: "Bebestibles");
40
41         Producto producto = new Producto();
42         producto.setSku(sku: "BEB001");
43         producto.setNombre(nombre: "Coca Cola");
44         producto.setDescripcion(descripcion: "Bebida lata 350ml");
45         producto.setPrecioBase(precioBase: 1000L);
46         producto.setGranel(granel: false);
47         producto.setCategoria(categoria);
48
49         when(productoRepository.save(any(Producto.class))).thenReturn(invocation -> {
50             Producto p = invocation.getArgument(0);

```

```

oServiceTest]/[method:existeProductoTrueTest()]
%TSTREE10, buscarProductoIdTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest), false, 1, false, 2, Debería
  buscar un producto por ID correctamente, [engine:junit-jupiter]/[class:com.punto_de_venta.service_catalogo.service.Prod
uctoServiceTest]/[method:buscarProductoIdTest()]
%TESTS 3, existeProductoFalseTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTE 3, existeProductoFalseTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTS 4, actualizarProductoTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTE 4, actualizarProductoTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTS 5, guardarProductoTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTE 5, guardarProductoTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTS 6, buscarProductoSkuTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTE 6, buscarProductoSkuTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTS 7, listarProductosTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTE 7, listarProductosTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTS 8, eliminarProductoTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTE 8, eliminarProductoTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTS 9, existeProductoTrueTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTE 9, existeProductoTrueTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTS 10, buscarProductoIdTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%TESTE 10, buscarProductoIdTest(com.punto_de_venta.service_catalogo.service.ProductoServiceTest)
%RUNTIME1284

```

Test Runner for Java

- ProductoServiceTest \$(symbol-name) com.punto_de_venta.servi
- actualizarProductoTest() \$(symbol-class) ProductoServiceTest + \$(sym
- buscarProductoIdTest() \$(symbol-class) ProductoServiceTest + \$(sym
- buscarProductoSkuTest() \$(symbol-class) ProductoServiceTest + \$(sym
- eliminarProductoTest() \$(symbol-class) ProductoServiceTest + \$(sym
- existeProductoFalseTest() \$(symbol-class) ProductoServiceTest + \$(sym
- existeProductoTrueTest() \$(symbol-class) ProductoServiceTest + \$(sym
- guardarProductoTest() \$(symbol-class) ProductoServiceTest + \$(sym
- listarProductosTest() \$(symbol-class) ProductoServiceTest + \$(sym

> 10 older results

6.3.2 Clase CategoriaServiceTest

Valida la lógica de categorías: creación, listado, búsqueda por ID, actualización y eliminación. Permite demostrar el ciclo básico del patrón Repository-Service.

EVIDENCIA: CODIFICACIÓN CategoriaServiceTest

```

23  @ExtendWith(MockitoExtension.class)
24  public class CategoriaServiceTest {
25
26      @Mock
27      private CategoriaRepository categoriaRepository;
28      @InjectMocks
29      private CategoriaService categoriaService;
30
31      @Test
32      @DisplayName("Debería guardar una categoría correctamente")
33
34      void guardarCategoriaTest(){
35
36          Categoria categoria = new Categoria();
37          categoria.setNombre(nombre: "Bebestibles");
38
39          when(categoriaRepository.save(any(Categoria.class))).thenReturn(invocation -> {
40              Categoria c = invocation.getArgument(0);
41              c.setId(id: 1L);
42              return c;
43          });
44
45          Categoria resultado = categoriaService.crearCategoria(categoria);

```

%TESTC 5 v2
 %TSTREE2,com.punto_de_venta.service_catalogo.service.CategoriaServiceTest,true,5,false,1,CategoriaServiceTest,,[engine:junit-jupiter]/[class:com.punto_de_venta.service_catalogo.service.CategoriaServiceTest]
 %TSTREE3,eliminarCategoriaTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest),false,1,false,2,Debería eliminar una categoría correctamente,,[engine:junit-jupiter]/[class:com.punto_de_venta.service_catalogo.service.CategoriaServiceTest]/[method:eliminarCategoriaTest()]
 %TSTREE4,actualizarCategoriaTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest),false,1,false,2,Debería actualizar una categoría correctamente,,[engine:junit-jupiter]/[class:com.punto_de_venta.service_catalogo.service.CategoriaServiceTest]/[method:actualizarCategoriaTest()]
 %TSTREE5,listaCategoriasTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest),false,1,false,2,Debería listar todas las categorías correctamente,,[engine:junit-jupiter]/[class:com.punto_de_venta.service_catalogo.service.CategoriaServiceTest]/[method:listaCategoriasTest()]
 %TSTREE6,buscarPorIdTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest),false,1,false,2,Debería buscar la categoría por Id correctamente,,[engine:junit-jupiter]/[class:com.punto_de_venta.service_catalogo.service.CategoriaServiceTest]/[method:buscarPorIdTest()]
 %TSTREE7,guardarCategoriaTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest),false,1,false,2,Debería guardar una categoría correctamente,,[engine:junit-jupiter]/[class:com.punto_de_venta.service_catalogo.service.CategoriaServiceTest]/[method:guardarCategoriaTest()]
 %TESTS 3,eliminarCategoriaTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest)
 %TESTE 3,eliminarCategoriaTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest)
 %TESTS 4,actualizarCategoriaTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest)
 %TESTE 4,actualizarCategoriaTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest)
 %TESTS 5,listaCategoriasTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest)
 %TESTE 5,listaCategoriasTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest)
 %TESTS 6,buscarPorIdTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest)
 %TESTE 6,buscarPorIdTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest)
 %TESTS 7,guardarCategoriaTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest)
 %TESTE 7,guardarCategoriaTest(com.punto_de_venta.service_catalogo.service.CategoriaServiceTest)
 %RUNTIME885

Test Runner for Java
 CategoriaServiceTest \$(symbol-namespace) com.punto_de_venta.service
 actualizarCategoriaTest() \$(symbol-class) CategoriaServiceTest + \$(symbol-name)
 buscarPorIdTest() \$(symbol-class) CategoriaServiceTest + \$(symbol-name)
 eliminarCategoriaTest() \$(symbol-class) CategoriaServiceTest + \$(symbol-name)
 guardarCategoriaTest() \$(symbol-class) CategoriaServiceTest + \$(symbol-name)
 listaCategoriasTest() \$(symbol-class) CategoriaServiceTest + \$(symbol-name)
 > 11 older results

6.3.3 Clase ClienteServiceTest

Valida la administración de clientes, incluyendo búsquedas por ID y RUN, actualización de datos de contacto y cambio de estado activo/inactivo.

EVIDENCIA PENDIENTE: CODIFICACIÓN ClienteServiceTest

```

23 @ExtendWith(MockitoExtension.class)
24 public class ClienteServiceTest {
25
26     @Mock
27     private ClienteRepository clienteRepository;
28
29     @InjectMocks
30     private ClienteService clienteService;
31
32     @Test
33     @DisplayName("Debería crear un cliente correctamente")
34
35     void guardarClienteTest(){
36
37         Cliente cliente = new Cliente();
38         cliente.setRun(run: 12345678L);
39         cliente.setDvrun(dvrun: '5');
40         cliente.setPNombre(pNombre: "Catalina");
41         cliente.setSNombre(sNombre: "Martina");
42         cliente.setApPaterno(apPaterno: "Gonzales");
43         cliente.setApMaterno(apMaterno: "Herrera");
44         cliente.setCorreo(correo: "catalina@gmail.com");
45         cliente.setTelefono(telefono: 984848484L);
46         cliente.setActivo(activo: true);

```

```
%$TSTREE2,com.punto_de_venta.service_cliente.service.ClienteServiceTest, true, 6, false, 1, ClienteServiceTest,, [engine:junit-jupiter]/[class:com.punto_de_venta.service_cliente.service.ClienteServiceTest]
%$TSTREE3,actualizarEstadoClienteTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest), false, 1, false, 2, Deberia actualizar el estado del cliente correctamente, [engine:junit-jupiter]/[class:com.punto_de_venta.service_cliente.service.ClienteServiceTest]/[method:actualizarEstadoClienteTest()]
%$TSTREE4, listarClientesTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest), false, 1, false, 2, Deberia listar los clientes correctamente, [engine:junit-jupiter]/[class:com.punto_de_venta.service_cliente.service.ClienteServiceTest]/[method:listarClientesTest()]
%$TSTREE5, actualizarClienteTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest), false, 1, false, 2, Deberia actualizar a un cliente correctamente, [engine:junit-jupiter]/[class:com.punto_de_venta.service_cliente.service.ClienteServiceTest]/[method:actualizarClienteTest()]
%$TSTREE6, guardarClienteTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest), false, 1, false, 2, Deberia crear un cliente correctamente, [engine:junit-jupiter]/[class:com.punto_de_venta.service_cliente.service.ClienteServiceTest]/[method:guardarClienteTest()]
%$TSTREE7, buscarClienteId(com.punto_de_venta.service_cliente.service.ClienteServiceTest), false, 1, false, 2, Deberia buscar al cliente por Id correctamente, [engine:junit-jupiter]/[class:com.punto_de_venta.service_cliente.service.ClienteServiceTest]/[method:buscarClienteId()]
%$TSTREE8, buscarClienteRunTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest), false, 1, false, 2, Deberia buscar a un cliente por run correctamente, [engine:junit-jupiter]/[class:com.punto_de_venta.service_cliente.service.ClienteServiceTest]/[method:buscarClienteRunTest()]
%$TESTS 3,actualizarEstadoClienteTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest)
%$TESTE 3,actualizarEstadoClienteTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest)
%$TESTS 4, listarClientesTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest)
%$TESTE 4, listarClientesTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest)
%$TESTS 5, actualizarClienteTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest)
%$TESTE 5, actualizarClienteTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest)
%$TESTS 6, guardarClienteTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest)
%$TESTE 6, guardarClienteTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest)
%$TESTS 7, buscarClienteId(com.punto_de_venta.service_cliente.service.ClienteServiceTest)
%$TESTE 7, buscarClienteId(com.punto_de_venta.service_cliente.service.ClienteServiceTest)
%$TESTS 8, buscarClienteRunTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest)
%$TESTE 8, buscarClienteRunTest(com.punto_de_venta.service_cliente.service.ClienteServiceTest)

%$RUNTIME$20
```

```
Test Runner for Java
  @ ClienteServiceTest $(symbol=nombre) com.punto_venta.service.c
  @ actualizarClienteTest() $(symbol=class) ClienteServiceTest + $(symbol=na
  @ actualizarEstadoClienteTest() $(symbol=class) ClienteServiceTest + $(sym
  @ buscarCliente() $(symbol=class) ClienteServiceTest + $(symbol=nombre)
  @ buscarClienteRunTest() $(symbol=class) ClienteServiceTest + $(symbol=na
  @ guardarClienteTest() $(symbol=class) ClienteServiceTest + $(symbol=na
  @ listarClientesTest() $(symbol=class) ClienteServiceTest + $(symbol=na
```

6.3.4 Clase FacturaServiceTest

Valida la generación de factura y el cálculo de valores tributarios. El método generarFactura calcula subtotal e impuesto a partir del monto total y guarda la entidad mediante FacturaRepository.

EVIDENCIA: CODIFICACIÓN FacturaServiceTest


```

22 @ExtendWith(MockitoExtension.class)
23 public class FacturaServiceTest {
24
25     @Mock
26     private FacturaRepository ftr;
27
28     @Mock
29     private WebClient.Builder webClientBuilder;
30
31     @InjectMocks
32     private FacturaService facturaService;
33
34     @Test
35     @DisplayName("Debería calcular el subtotal e impuestos correctamente y guardar la factura")
36     void generarFactura(){
37         Long idOrdenSimulada = 101L;
38         Long montoTotalSimulado = 1190L;
39
40         when(ftr.save(any(Factura.class))).thenAnswer(invocation -> invocation.getArgument(0));
41
42         Factura resultado = facturaService.generarFactura(idOrdenSimulada, montoTotalSimulado);
43
44         assertNotNull(resultado, "La factura generada no debe ser nula");
45         assertEquals(idOrdenSimulada, resultado.getIdOrden());
46
47         assertEquals(1000L, resultado.getMontoSubtotal(), "El monto subtotal calculado es incorrecto");
48
49         assertEquals(new BigDecimal("190"), resultado.getImpuesto(), "El impuesto calculado es incorrecto");
50         assertEquals(montoTotalSimulado, resultado.getMontoTotal(), "El monto total debe mantenerse igual");
51         assertNotNull(resultado.getFechaEmision(), "La factura debe tener fecha de emisión");
52
53         verify(ftr, times(1)).save(any(Factura.class));
54     }
55
56     @Test
57     @DisplayName("Debería listar todas las facturas")
58     void listarFacturasTest(){
59         Factura f1 = new Factura();
60         Factura f2 = new Factura();
61         List<Factura> listaSimulada = List.of(f1, f2);
62
63         when(ftr.findAll()).thenReturn(listaSimulada);

```

```
%TESTE 5 v2
%$TSTREE2,com.punto_de_venta.service_facturacion.service.FacturaServiceTest,true,5,false,1,FacturaServiceTest,,[engine:junit-jupiter]/[class:com.punto_de_venta.service_facturacion.service.FacturaServiceTest]
%$TSTREE3,listarFacturasPorFechaTest(com.punto_de_venta.service_facturacion.service.FacturaServiceTest),false,1,false,2,Deberia listar facturas ordenadas por fecha,[engine:junit-jupiter]/[class:com.punto_de_venta.service_facturacion.service.FacturaServiceTest]/[method:listarFacturasPorFechaTest()]
%$TSTREE4,buscarFacturaPorId_NoExiste_Test(com.punto_de_venta.service_facturacion.service.FacturaServiceTest),false,1,false,2,Deberia retornar nulo si la factura no existe,[engine:junit-jupiter]/[class:com.punto_de_venta.service_facturacion.service.FacturaServiceTest]/[method:buscarFacturaPorId_NoExiste_Test()]
%$TSTREE5,listarFacturasTest(com.punto_de_venta.service_facturacion.service.FacturaServiceTest),false,1,false,2,Deberia listar todas las facturas,[engine:junit-jupiter]/[class:com.punto_de_venta.service_facturacion.service.FacturaServiceTest]/[method:listarFacturasTest()]
%$TSTREE6,buscarFacturaPorId_Existe_Test(com.punto_de_venta.service_facturacion.service.FacturaServiceTest),false,1,false,2,Deberia retornar una factura si el ID existe,[engine:junit-jupiter]/[class:com.punto_de_venta.service_facturacion.service.FacturaServiceTest]/[method:buscarFacturaPorId_Existe_Test()]
%$TSTREE7,generarFactura(com.punto_de_venta.service_facturacion.service.FacturaServiceTest),false,1,false,2,Deberia calcular el subtotal e impuestos correctamente y guardar la factura,[engine:junit-jupiter]/[class:com.punto_de_venta.service_facturacion.service.FacturaServiceTest]/[method:generarFactura()]
%TESTS 3,listarFacturasPorFechaTest(com.punto_de_venta.service_facturacion.service.FacturaServiceTest)
%TESTE 3,listarFacturasPorFechaTest(com.punto_de_venta.service_facturacion.service.FacturaServiceTest)
%TESTS 4,buscarFacturaPorId_NoExiste_Test(com.punto_de_venta.service_facturacion.service.FacturaServiceTest)
%TESTE 4,buscarFacturaPorId_NoExiste_Test(com.punto_de_venta.service_facturacion.service.FacturaServiceTest)
%TESTS 5,listarFacturasTest(com.punto_de_venta.service_facturacion.service.FacturaServiceTest)
%TESTE 5,listarFacturasTest(com.punto_de_venta.service_facturacion.service.FacturaServiceTest)
%TESTS 6,buscarFacturaPorId_Existe_Test(com.punto_de_venta.service_facturacion.service.FacturaServiceTest)
%TESTE 6,buscarFacturaPorId_Existe_Test(com.punto_de_venta.service_facturacion.service.FacturaServiceTest)
%TESTS 7,generarFactura(com.punto_de_venta.service_facturacion.service.FacturaServiceTest)
%TESTE 7,generarFactura(com.punto_de_venta.service_facturacion.service.FacturaServiceTest)
%RUNTIME1127
```

6.3.5 Clase ReporteServiceTest

Valida la generación de reportes consolidados. Se mockea WebClient para simular respuestas de ventas e inventario, evitando llamadas reales entre microservicios durante la prueba.

EVIDENCIA: CODIFICACIÓN ReporteServiceTest

```

public class ReporteServiceTest {
    void obtenerHistorialTest(){
        HistorialReporte reporte2 = new HistorialReporte();
        List<HistorialReporte> listaSimulada = Arrays.asList(reporte1, reporte2);

        when(repositorio.findAll()).thenReturn(listaSimulada);

        List<HistorialReporte> resultado = service.obtenerHistorial();

        assertNotNull(resultado, "La lista no debe ser nula");
        assertEquals(2, resultado.size(), "Debe retornar los dos elementos simulados");
        verify(repositorio, times(1)).findAll();
    }

    @Test
    @DisplayName("Debería generar un reporte consolidado consultando a otros microservicios")
    @SuppressWarnings("unchecked")
    void generarReporteConsolidadoTest(){
        String tipo = "MENSUAL";
        String usuario = "SYSTEM";
        Long valorSimuladoMicroservicios=1500L;

        WebClient webClient = Mockito.mock(WebClient.class);
        WebClient.RequestHeadersUriSpec uriSpec = Mockito.mock(WebClient.RequestHeadersUriSpec.class);
        WebClient.RequestHeadersSpec headersSpec = Mockito.mock(WebClient.RequestHeadersSpec.class);
        WebClient.ResponseSpec responseSpec = Mockito.mock(WebClient.ResponseSpec.class);

        when(webClientBuilder.build()).thenReturn(webClient);
        when(webClient.get()).thenReturn(uriSpec);
        when(uriSpec.uri(anyString())).thenReturn(headersSpec);
        when(headersSpec.retrieve()).thenReturn(responseSpec);

        when(responseSpec.bodyToMono(Long.class)).thenReturn(Mono.just(valorSimuladoMicroservicios));

        when(repositorio.save(any(HistorialReporte.class))).thenAnswer(invocation -> {
            HistorialReporte r = invocation.getArgument(0);
            return r;
        });

        HistorialReporte resultado = service.generarReporteConsolidado(tipo, usuario);

        assertNotNull(resultado, "el reporte generado no puede ser nulo");
        assertEquals(tipo, resultado.getTipoReporte(), "El tipo de reporte debe coincidir");
        assertEquals(usuario, resultado.getGeneradoPor(), "El usuario debe coincidir");

        assertEquals(1500L, resultado.getTotalVentasCalculadas());
        assertEquals(1500L, resultado.getAlertasStockDetectadas());
        assertNotNull(resultado.getFechaGeneracion(), "Debe tener una fecha asignada");
    }
}

```

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

```

%TESTC 2 v2
%TSTT2E2,com.punto_de_venta.service_reportes.service.ReporteServiceTest,true,2,false,1,ReporteServiceTest,,[engine:junit-jupiter]/[class:com.punto_de_venta.service_reportes.service.ReporteServiceTest]
%TSTT3E3,obtenerHistorialTest(com.punto_de_venta.service_reportes.service.ReporteServiceTest),false,1,false,2,Debería obtener el historial de reporte correctamente,,[engine:junit-jupiter]/[class:com.punto_de_venta.service_reportes.service.ReporteServiceTest]/[method:obtenerHistorialTest()]
%TSTT4E4,generarReporteConsolidadoTest(com.punto_de_venta.service_reportes.service.ReporteServiceTest),false,1,false,2,Debería generar un reporte consolidado consultando a otros microservicios,,[engine:junit-jupiter]/[class:com.punto_de_venta.service_reportes.service.ReporteServiceTest]/[method:generarReporteConsolidadoTest()]
%TESTS 3,obtenerHistorialTest(com.punto_de_venta.service_reportes.service.ReporteServiceTest)

%TESTE 3,obtenerHistorialTest(com.punto_de_venta.service_reportes.service.ReporteServiceTest)

%TESTS 4,generarReporteConsolidadoTest(com.punto_de_venta.service_reportes.service.ReporteServiceTest)

%TESTE 4,generarReporteConsolidadoTest(com.punto_de_venta.service_reportes.service.ReporteServiceTest)

%RUNTIME1361

```

Test Runner for Java

- ReporteServiceTest \$(symbol-namespace) com.punto_de_venta.service_reportes
- generarReporteConsolidadoTest() \$(symbol-class) ReporteServiceTest
- obtenerHistorialTest() \$(symbol-class) ReporteServiceTest

14 older results

6.3.6 Clase ProveedorServiceTest

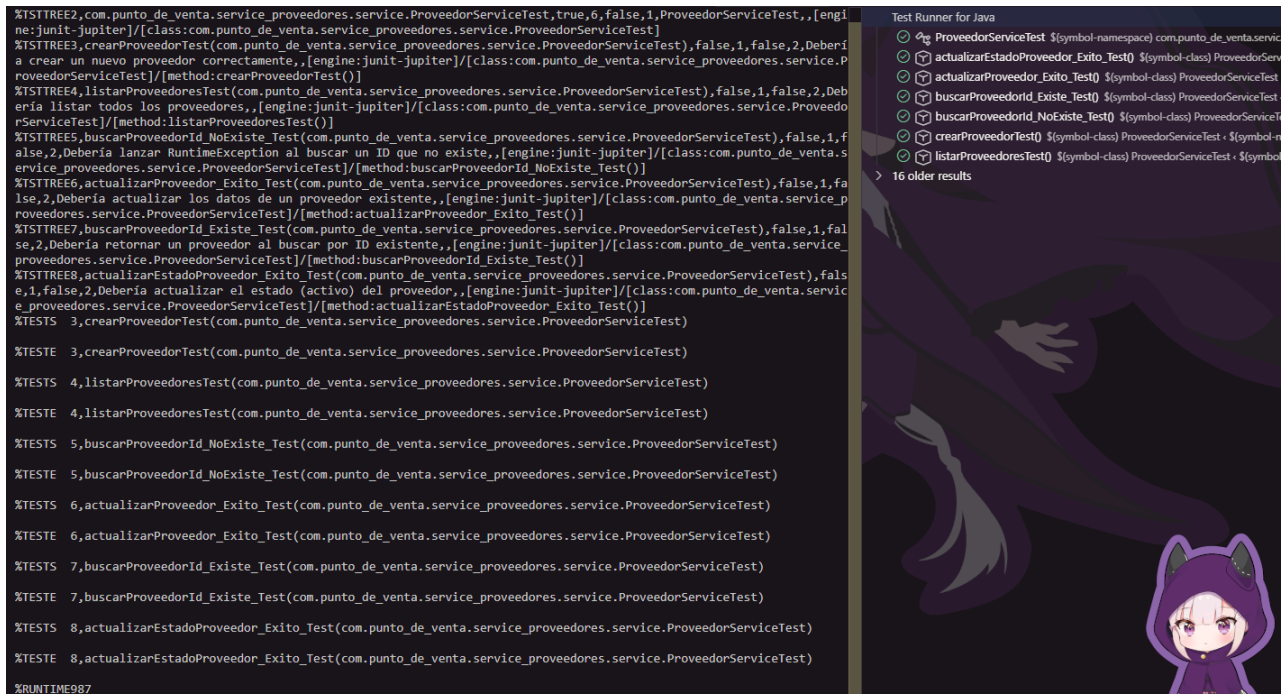
Valida la administración de proveedores, incluyendo creación, listado, búsqueda, manejo de error, actualización de datos y actualización de estado.

EVIDENCIA: CODIFICACIÓN ProveedorServiceTest

```

20  @ExtendWith(MockitoExtension.class)
21  class ProveedorServiceTest {
22
23      @Mock
24      private ProveedorRepository proveedorRepository;
25
26      @InjectMocks
27      private ProveedorService proveedorService;
28
29      @Test
30      @DisplayName("Debería crear un nuevo proveedor correctamente")
31      void crearProveedorTest() {
32          Proveedor nuevoProveedor = new Proveedor();
33          nuevoProveedor.setNombreContacto(nombreContacto: "Juan Perez");
34
35          when(proveedorRepository.save(any(Proveedor.class))).thenReturn(nuevoProveedor);
36
37          Proveedor resultado = proveedorService.crearProveedor(nuevoProveedor);
38
39          assertNotNull(resultado, "El proveedor creado no debería ser nulo");
40          assertEquals("Juan Perez", resultado.getNombreContacto(), "El nombre del contacto debe coincidir");
41
42          verify(proveedorRepository, times(1)).save(any(Proveedor.class));
43      }
44
45      @Test
46      @DisplayName("Debería listar todos los proveedores")
47      void listarProveedoresTest() {
48          List<Proveedor> listaSimulada = List.of(new Proveedor(), new Proveedor());
49          when(proveedorRepository.findAll()).thenReturn(listaSimulada);
50
51          List<Proveedor> resultado = proveedorService.listarProveedores();
52
53          assertNotNull(resultado);
54          assertEquals(2, resultado.size(), "Debería retornar 2 proveedores");
55          verify(proveedorRepository, times(1)).findAll();
56      }
57
58      @Test
59      @DisplayName("Debería retornar un proveedor al buscar por ID existente")

```



```

%TSTREE2,com.punto_de_venta.service.proveedores.service.ProveedorServiceTest,true,6,false,1,ProveedorServiceTest,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ProveedorServiceTest]
%TSTREE3,crearProveedorTest(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest),false,1,false,2,Debería crear un nuevo proveedor correctamente,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ProveedorServiceTest]/[method:crearProveedorTest()]
%TSTREE4,listarProveedoresTest(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest),false,1,false,2,Debería listar todos los proveedores,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ProveedorServiceTest]/[method:listarProveedoresTest()]
%TSTREE5,buscarProveedorId_NoExiste_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest),false,1,false,2,Debería lanzar RuntimeException al buscar un ID que no existe,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ProveedorServiceTest]/[method:buscarProveedorId_NoExiste_Test()]
%TSTREE6,actualizarProveedor_Exit_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest),false,1,false,2,Debería actualizar los datos de un proveedor existente,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ProveedorServiceTest]/[method:actualizarProveedor_Exit_Test()]
%TSTREE7,buscarProveedorId_Exist_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest),false,1,false,2,Debería retornar un proveedor al buscar por ID existente,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ProveedorServiceTest]/[method:buscarProveedorId_Exist_Test()]
%TSTREE8,actualizarEstadoProveedor_Exit_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest),false,1,false,2,Debería actualizar el estado (activo) del proveedor,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ProveedorServiceTest]/[method:actualizarEstadoProveedor_Exit_Test()]
%TESTS 3,crearProveedorTest(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%TESTE 3,crearProveedorTest(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%TESTS 4,listarProveedoresTest(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%TESTE 4,listarProveedoresTest(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%TESTS 5,buscarProveedorId_NoExiste_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%TESTE 5,buscarProveedorId_NoExiste_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%TESTS 6,actualizarProveedor_Exit_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%TESTE 6,actualizarProveedor_Exit_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%TESTS 7,buscarProveedorId_Exist_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%TESTE 7,buscarProveedorId_Exist_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%TESTS 8,actualizarEstadoProveedor_Exit_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%TESTE 8,actualizarEstadoProveedor_Exit_Test(com.punto_de_venta.service.proveedores.service.ProveedorServiceTest)

%RUNTIME987

```

6.3.7 Clase ItemCompraServiceTest

Valida que al crear un ítem de compra se calcule el subtotal, se actualice el costo total de la orden y se guarde el ítem.

EVIDENCIA: CODIFICACIÓN ItemCompraServiceTest

```

23 @ExtendWith(MockitoExtension.class)
24 class ItemCompraServiceTest {
25
26     @Mock
27     private ItemCompraRepository itemCompraRepository;
28
29     @Mock
30     private OrdenCompraRepository ordenCompraRepository;
31
32     @Mock
33     private OrdenCompraService ordenCompraService;
34
35     @InjectMocks
36     private ItemCompraService itemCompraService;
37
38     @Test
39     @DisplayName("Debería calcular el subtotal, actualizar la orden y guardar el ItemCompra")
40     void crearItemCompraTest() {
41         OrdenCompra ordenSimulada = new OrdenCompra();
42         ordenSimulada.setId(id: 10L);
43         ordenSimulada.setCostoTotal(costoTotal: 500L);
44         ItemCompra itemEntrante = new ItemCompra();
45         itemEntrante.setCantidad(new BigDecimal("2"));
46         itemEntrante.setCostoUnitario(costoUnitario: 150L);
47         itemEntrante.setOrdenCompra(ordenSimulada);
48
49         when(ordenCompraService.buscarOrdenCompraId(id: 10L)).thenReturn(ordenSimulada);
50
51         when(ordenCompraRepository.save(any(OrdenCompra.class))).thenAnswer(i -> i.getArgument(0));
52         when(itemCompraRepository.save(any(ItemCompra.class))).thenAnswer(i -> i.getArgument(0));
53
54         ItemCompra resultado = itemCompraService.crearItemCompra(itemEntrante);
55
56         assertNotNull(resultado, "El ItemCompra no debe ser nulo");
57
58         assertEquals(800L, resultado.getOrdenCompra().getCostoTotal(), "El costo total de la orden no se sumó correctamente");
59
60         verify(ordenCompraService, times(1)).buscarOrdenCompraId(id: 10L);
61         verify(ordenCompraRepository, times(1)).save(any(OrdenCompra.class));
62         verify(itemCompraRepository, times(1)).save(itemEntrante);
63

```

PROBLEMS 121 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

```

%TESTC 4 v2
%TSTREE2,com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest,true,4,false,1,ItemCompraServiceTest,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest]
%TSTREE3,buscarItemCompraId_NoExiste_Test(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest),false,1,false,2,Debería lanzar RuntimeException al buscar un ID que no existe,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest]/[method:buscarItemCompraId_NoExiste_Test()]
%TSTREE4,crearItemCompraTest(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest),false,1,false,2,Debería calcular el subtotal, actualizar la orden y guardar el ItemCompra,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest]/[method:crearItemCompraTest()]
%TSTREE5,listaItemsCompraTest(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest),false,1,false,2,Debería listar todos los ítems de compra,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest]/[method:listaItemsCompraTest()]
%TSTREE6,buscarItemCompraId_Existe_Test(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest),false,1,false,2,Debería retornar un ítem al buscar por ID existente,,[engine:junit-jupiter]/[class:com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest]/[method:buscarItemCompraId_Existe_Test()]
%TESTS 3,buscarItemCompraId_NoExiste_Test(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest)
%TESTE 3,buscarItemCompraId_NoExiste_Test(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest)
%TESTS 4,crearItemCompraTest(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest)
%TESTE 4,crearItemCompraTest(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest)
%TESTS 5,listaItemsCompraTest(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest)
%TESTE 5,listaItemsCompraTest(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest)
%TESTS 6,buscarItemCompraId_Existe_Test(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest)
%TESTE 6,buscarItemCompraId_Existe_Test(com.punto_de_venta.service.proveedores.service.ItemCompraServiceTest)
%RUNTIME1190

```

Test Runner for Java

- ItemCompraServiceTest (symbol-namespace) com.punto_de_venta.serv
- buscarItemCompraId_Existe_Test() (symbol-class) ItemCompraService
- buscarItemCompraId_NoExiste_Test() (symbol-class) ItemCompraServ
- crearItemCompraTest() (symbol-class) ItemCompraServiceTest + (sym
- listaItemsCompraTest() (symbol-class) ItemCompraServiceTest + (sym

15 older results

6.3.8 Clase OrdenCompraServiceTest

Valida la creación de órdenes de compra, listado con carga de productos vía WebClient y cambio de estado a RECIBIDA, gatillando movimiento de inventario.

EVIDENCIA: CODIFICACIÓN OrdenCompraServiceTest

```

29  @ExtendWith(MockitoExtension.class)
30  class OrdenCompraServiceTest {
31
32      @Mock
33      private OrdenCompraRepository ordenCompraRepository;
34
35      @Mock
36      private ProveedorService proveedorService;
37
38      @Mock
39      private WebClient.Builder webClientBuilder;
40
41      @InjectMocks
42      private OrdenCompraService ordenCompraService;
43
44      @Test
45      @DisplayName("Debería crear una orden de compra iniciando su costo en 0")
46      void crearOrdenTest() {
47          Proveedor proveedorSimulado = new Proveedor();
48          proveedorSimulado.setId(id: 1L);
49
50          OrdenCompra nuevaOrden = new OrdenCompra();
51          nuevaOrden.setProveedor(proveedorSimulado);
52
53          when(proveedorService.buscarProveedorId(id: 1L)).thenReturn(proveedorSimulado);
54
55          when(ordenCompraRepository.save(any(OrdenCompra.class))).thenReturn(i -> i.getArgument(0));
56
57          OrdenCompra resultado = ordenCompraService.crearOrden(nuevaOrden);
58
59          assertNotNull(resultado);
60          assertEquals(0L, resultado.getCostoTotal(), "El costo inicial debe setearse en 0L");
61
62          verify(proveedorService, times(1)).buscarProveedorId(id: 1L);
63          verify(ordenCompraRepository, times(1)).save(any(OrdenCompra.class));
64      }

```

PROBLEMS 121 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

```

%TESTC 3 v2
%TSTTREE2,com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTest,true,3,false,1,OrdenCompraServiceTest,,[
engine:junit-jupiter]/[class:com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTest]
%TSTTREE3,actualizarStatusOrdenCompra_Recibida_Test(com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTes
t),false,1,false,2,Debería actualizar el estado de la orden y gatillar un movimiento de inventario si pasa a RECIBIDA,,[
engine:junit-jupiter]/[class:com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTest]/[method:actualizarSt
atusOrdenCompra_Recibida_Test()]
%TSTTREE4,listarOrdenesTest(com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTest),false,1,false,2,Deber
ía listar las órdenes e intentar cargar los datos de los productos mediante WebClient,,[engine:junit-jupiter]/[class:com
.punto_de_venta.service_proveedores.service.OrdenCompraServiceTest]/[method:listarOrdenesTest()]
%TSTTREE5,crearOrdenTest(com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTest),false,1,false,2,Debería
crear una orden de compra iniciando su costo en 0,,[engine:junit-jupiter]/[class:com.punto_de_venta.service_proveedores
.service.OrdenCompraServiceTest]/[method:crearOrdenTest()]
%TESTS 3,actualizarStatusOrdenCompra_Recibida_Test(com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTes
t)
%TESTE 3,actualizarStatusOrdenCompra_Recibida_Test(com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTes
t)
%TESTS 4,listarOrdenesTest(com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTest)
%TESTE 4,listarOrdenesTest(com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTest)
%TESTS 5,crearOrdenTest(com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTest)
%TESTE 5,crearOrdenTest(com.punto_de_venta.service_proveedores.service.OrdenCompraServiceTest)
%RUNTIME1362

```

Test Runner for Java

- OrdenCompraServiceTest \$(symbol-namespace) com.punto_de_venta.se
- actualizarStatusOrdenCompra_Recibida_Test \$(symbol-class) Orden...
- crearOrdenTest \$(symbol-class) OrdenCompraServiceTest + \$(symbol-na
- listarOrdenesTest \$(symbol-class) OrdenCompraServiceTest + \$(symbol-na

> 17 older results

7. Anexo: rutas y cuerpos de prueba sugeridos

Los siguientes ejemplos sirven para completar las capturas de Postman. Cuando se pruebe a través del gateway, usar `http://localhost:4425` y agregar `Authorization: Bearer <token>` en los endpoints protegidos.

Servicio	Método	URL	Body / nota
Auth	POST	<code>http://localhost:4425/api/v1/auth/login</code>	<code>{"nombreUsuario":"servicios","contrasena":"Servicios123"}</code>
Categoría	POST	<code>http://localhost:4425/api/v1/categorias</code>	<code>{"nombre":"Abarrotes"}</code>
Producto	POST	<code>http://localhost:4425/api/v1/productos</code>	<code>{"sku":"ARR-001","nombre":"Arroz","descripcion":"Arroz 1kg","precioBase":1200,"granel":false,"categoria":{"id":1}}</code>
Cliente	POST	<code>http://localhost:4425/api/v1/clientes</code>	<code>{"run":12345678,"dvrn":9,"pnombre":"Carolina","snombre":"Andrea","apMaterno":"Baeza","apPaterno":"Hermosilla","correo":"cliente@correo.cl","telefono":987654321,"activo":true}</code>
Stock	GET	<code>http://localhost:4425/api/v1/stock</code>	Listar stock.
Movimiento inventario	POST	<code>http://localhost:4425/api/v1/movimiento-inventario/nuevo/1</code>	<code>{"tipoMovimiento":"ENTRADA","monto":10}</code>
Proveedor	POST	<code>http://localhost:4425/api/v1/proveedores</code>	<code>{"compañia":"Distribuidora Central","nombreContacto":"Ana Pérez","email":"contacto@proveedor.cl","activo":true}</code>
Orden compra	POST	<code>http://localhost:4425/api/v1/ordenes-compras</code>	<code>{"proveedor":{"id":1},"status":"PENDIENTE","fechaCompra":"2026-06-21"}</code>
Item compra	POST	<code>http://localhost:4425/api/v1/items-compras</code>	<code>{"ordenCompra":{"id":1},"idProducto":1,"cantidad":10,"costoUnitario":900,"granel":false}</code>
Factura	POST	<code>http://localhost:4425/api/v1/facturas/generar</code>	<code>{"idOrden":1,"montoTotal":1190}</code>
Reporte	POST/GET	<code>http://localhost:4425/api/v1/reportes</code>	Ver método exacto en <code>ReporteController</code> y Swagger.
Notificación	POST	<code>http://localhost:4425/api/v1/notificaciones</code>	<code>{"email":"cliente@correo.cl","tipo":"Compra registrada","status":"Enviada","fechaEnvio":"2026-06-21"}</code>

8. Conclusión

La Experiencia 3 permitió consolidar una arquitectura de microservicios funcional, documentada y protegida mediante autenticación por token. La separación por capas facilita el mantenimiento del código y permite que cada microservicio tenga responsabilidades claras. El API Gateway cumple un rol central al unificar el acceso y proteger las rutas mediante JWT. Swagger/OpenAPI mejora la trazabilidad y validación de endpoints, mientras que las pruebas unitarias con JUnit y Mockito entregan evidencia técnica de que la lógica principal funciona de forma aislada. Finalmente, la comunicación mediante WebClient permite integrar procesos distribuidos como ventas, inventario, proveedores, pagos, facturación y reportes sin mezclar directamente las bases de datos de los servicios.